prompttools

Developer Tools for LLM Prompts

Lint, format, test, and estimate costs for your LLM prompt files

---

Documentation v1.0.0

Author: Scott Converse

License: MIT

Python 3.9+

Table of Contents

1. Overview
2. Installation
3. Getting Started
4. promptools-core Reference
5. promptfmt Reference
6. promptcost Reference
7. promptest Reference
8. Configuration Reference
9. CI/CD Integration Guide
10. Plugin Development Guide
11. Troubleshooting
12. FAQ

Overview

prompttools is a monorepo containing five developer tools that treat LLM prompts as first-class code artifacts. Think eslint/prettier/jest, but for prompts.

Package	Version	Description
prompttools-core	1.0.0	Shared foundation: parsing, tokenization, model profiles, config, cache, plugins
promptlint	0.3.0	Static analysis and linting for prompt files with fixable rules
promptfmt	1.0.0	Auto-formatter (whitespace, delimiters, variables, wrapping, structure)
promptcost	1.0.0	Cost estimation, model comparison, volume projections, budget enforcement
promptest	1.0.0	Test framework with 15 assertion types and CI-ready output formats

All tools support four prompt file formats: YAML (`.yaml` / `.yml`), JSON (`.json`), Markdown (`.md`), and plain text (`.txt`).

Template variables are detected in three syntaxes: `{{var}}` (Jinja), `{var}` (f-string), and `<var>` (XML/angle bracket).

Installation

Install Individual Packages

```
pip install promptfmt           # auto-formatter
pip install promptcost          # cost estimator
pip install promptlint          # linter
```

Each tool package depends on `promptfmt` and will install it automatically.

Development Installation (Monorepo)

```
git clone <repository-url>
cd promptfmt

pip install -e packages/promptfmt[dev]
pip install -e packages/promptcost[dev]
pip install -e packages/promptlint[dev]
```

Running Tests

```
# All tests
pytest

# Single package
pytest packages/promptfmt/tests/
pytest packages/promptcost/tests/
pytest packages/promptlint/tests/

# Linting and type checking
ruff check .
mypy packages/
```

System Requirements

- Python 3.9, 3.10, 3.11, or 3.12
- Key dependencies: pydantic >= 2.0, pyyaml >= 6.0, tiktoken >= 0.7, typer >= 0.12, rich >= 13.0

Getting Started

Step 1: Create a prompt file

Create `prompts/greeting.yaml`:

```yaml
model: gpt-4o
description: A friendly greeting prompt
messages:
  - role: system
    content: You are a helpful assistant that greets users warmly.
  - role: user
    content: Hello! My name is {{user_name}}.
```

Step 2: Parse and inspect it

```python
from promptools_core import parse_file, Tokenizer

prompt = parse_file("prompts/greeting.yaml")
print(f"Format: {prompt.format.value}")
print(f"Messages: {len(prompt.messages)}")
print(f"Variables: {list(prompt.variables.keys())}")
print(f"Metadata: {prompt.metadata}")

tokenizer = Tokenizer.for_model("gpt-4o")
total = tokenizer.count_file(prompt)
print(f"Total tokens: {total}")
```

Step 3: Format the prompt file

```
promptfmt format prompts/greeting.yaml
```

Step 4: Estimate cost

```
promptcost estimate prompts/greeting.yaml --model gpt-4o --project 1000/day
```

Step 5: Write tests

Create `tests/test_greeting.yaml`:

```yaml
suite: greeting-tests
prompt: ../prompts/greeting.yaml
model: gpt-4o

tests:
  - name: has-system-message
    assert: has_role
```

```yaml
    role: system

  - name: uses-user-name-variable
    assert: has_variables
    variables:
      - user_name

  - name: token-count-reasonable
    assert: max_tokens
    max: 2048

  - name: no-injection-risk
    assert: not_contains
    text: "ignore previous instructions"

  - name: cost-under-budget
    assert: max_cost
    max: 0.05
```

Run the tests:

```
promptttest run tests/test_greeting.yaml
```

prompttools-core Reference

Parsing

`parse_file(path, config=None) -> PromptFile`

Parse a prompt file, auto-detecting format by extension (`.yaml` , `.yml` , `.json` , `.md` , `.txt`).

```
from prompttools_core import parse_file
prompt = parse_file("prompts/greeting.yaml")
```

Raises `ParseError` for unsupported formats and `FileNotFoundError` if the file does not exist.

`parse_directory(path, config=None, patterns=None) -> list[PromptFile]`

Recursively parse all prompt files in a directory. Skips files that fail to parse. Respects `config.exclude` patterns.

```
from prompttools_core import parse_directory
prompts = parse_directory("prompts/")
```

`parse_stdin(content, format) -> PromptFile`

Parse prompt content from stdin. The `format` parameter must be one of: `text` , `md` , `yaml` , `json` . The resulting `PromptFile` has `path` set to `Path("-")` .

`parse_pipeline(manifest_path, config=None) -> PromptPipeline`

Parse a pipeline manifest YAML and all referenced prompt files.

Pipeline manifest format:

```yaml
name: my-pipeline
model: gpt-4o
stages:
  - name: extract
    file: prompts/extract.yaml
    expected_output_tokens: 500
  - name: summarize
    file: prompts/summarize.yaml
    depends_on: [extract]
    persona: summarizer
```

Tokenization

`Tokenizer` class

```
from prompttools_core import Tokenizer

# Default tokenizer (cl100k_base encoding)
tokenizer = Tokenizer()

# Model-aware tokenizer
```

```
tokenizer = Tokenizer.for_model("gpt-4o")

# Count tokens in a string
count = tokenizer.count("Hello, world!")

# Count tokens for a full prompt file
total = tokenizer.count_file(prompt)

# Count tokens across messages including per-message role overhead
total = tokenizer.count_messages(prompt.messages)
```

Role overhead per provider: OpenAI adds 4 tokens per message, Anthropic adds 3, Google adds 3.

`count_tokens(text, encoding="cl100k_base") -> int`

Standalone convenience function.

`get_encoding(name="cl100k_base")`

Returns a cached tiktoken encoding. Results are LRU-cached (up to 8 encodings).

Model Profiles

Built-in Profiles

Model	Provider	Context Window	Input $/Mtok	Output $/Mtok
gpt-4	openai	8,192	$30.00	$60.00
gpt-4-turbo	openai	128,000	$10.00	$30.00
gpt-4o	openai	128,000	$2.50	$10.00
gpt-4o-mini	openai	128,000	$0.15	$0.60
claude-3-haiku	anthropic	200,000	$0.25	$1.25
claude-3-sonnet	anthropic	200,000	$3.00	$15.00
claude-3-opus	anthropic	200,000	$15.00	$75.00
claude-4-sonnet	anthropic	200,000	$3.00	$15.00
gemini-1.5-pro	google	1,000,000	$1.25	$5.00
gemini-2.0-flash	google	1,048,576	$0.10	$0.40

Non-OpenAI models use cl100k_base as an approximation since their native tokenizers are not publicly available via tiktoken.

Profile API

```
from promptttools_core import get_profile, list_profiles, register_profile, ModelProfile

# Look up a profile
profile = get_profile("gpt-4o")
```

```python
# List all profiles (built-in + custom)
all_profiles = list_profiles()

# Register a custom profile
register_profile(ModelProfile(
    name="my-custom-model",
    context_window=32000,
    encoding="cl100k_base",
    provider="custom",
    input_price_per_mtok=1.0,
    output_price_per_mtok=3.0,
    max_output_tokens=4096,
    supports_tools=True,
))
```

Custom profiles take precedence over built-in profiles with the same name.

Caching

```python
from promptools_core import PromptCache
from pathlib import Path

cache = PromptCache(cache_dir=Path(".promptools-cache"))

# Generate a content-based key
key = PromptCache.content_key(content="Hello", encoding="cl100k_base")

# Store a value (with optional TTL in seconds)
cache.set(key, {"tokens": 42}, ttl=3600)

# Retrieve (returns None if missing or expired)
value = cache.get(key)

# Invalidate a single key
cache.invalidate(key)

# Clear entire cache directory
cache.clear()
```

The cache stores entries as JSON in `.promptools-cache/cache.json`. Each entry includes a timestamp. TTL expiry is checked on `get()`.

Plugin System

```python
from promptools_core import discover_plugins, load_plugin
from pathlib import Path

# Load plugin classes from a single Python file
classes = load_plugin(Path("my_plugin.py"), base_class=MyBaseClass)
```

```
# Discover all plugins across multiple directories
classes = discover_plugins(
    plugin_dirs=[Path("./plugins")],
    base_class=MyBaseClass,
)
```

Plugin files must be `.py` files (not starting with `_`). Each file is loaded as a module and scanned for concrete subclasses of the given base class. Duplicate class names across files are skipped with a warning.

Data Models

PromptFile

Field	Type	Description
`path`	`Path`	Source file path
`format`	`PromptFormat`	TEXT, MARKDOWN, YAML, or JSON
`raw_content`	`str`	Original unmodified content
`messages`	`list[Message]`	Parsed messages
`variables`	`dict[str, str]`	Variable name to syntax style mapping
`variable_defaults`	`dict[str, str]`	Default values from metadata
`metadata`	`dict[str, Any]`	Front-matter or top-level metadata
`total_tokens`	`Optional[int]`	Populated by tokenizer
`content_hash`	`str`	SHA256 hash of raw_content

Helper methods: `system_message()`, `user_messages()`, `has_role(role)`.

Message

Field	Type	Description
`role`	`Literal["system", "user", "assistant", "tool"]`	Message role
`content`	`str`	Text content
`line_start` / `line_end`	`Optional[int]`	Source line numbers
`token_count`	`Optional[int]`	Populated by tokenizer
`metadata`	`dict[str, Any]`	Per-message metadata

ModelProfile

Field	Type	Description
`name`	`str`	Model identifier
`context_window`	`int`	Max context in tokens

`encoding`	`str`	tiktoken encoding name
`provider`	`str`	openai, anthropic, google, custom
`input_price_per_mtok`	`Optional[float]`	USD per million input tokens
`output_price_per_mtok`	`Optional[float]`	USD per million output tokens
`max_output_tokens`	`Optional[int]`	Max output tokens
`supports_system_message`	`bool`	Supports system messages
`supports_tools`	`bool`	Supports tool use
`approximate_tokenizer`	`bool`	Tokenizer is approximate

ToolConfig

Field	Type	Default	Description
`model`	`Optional[str]`	`None`	Model profile name
`tokenizer_encoding`	`Optional[str]`	`None`	Override encoding
`exclude`	`list[str]`	`[]`	Glob patterns to exclude
`plugins`	`list[str]`	`[]`	Plugin directory paths
`cache_enabled`	`bool`	`False`	Enable caching
`cache_dir`	`Path`	`.promptools-cache`	Cache directory
`extra`	`dict[str, Any]`	`{}`	Tool-specific config

Error Hierarchy

All exceptions inherit from `PromptToolsError`:

Exception	Description
`ParseError`	File cannot be parsed (also a `ValueError`)
`ConfigError`	Configuration is invalid
`TokenizerError`	Tokenization fails
`ProfileNotFoundError`	Unknown model profile
`PluginError`	Plugin loading/execution fails
`CacheError`	Cache read/write fails

promptfmt Reference

CLI Commands

`promptfmt format <path>`

Format prompt files in-place or check formatting.

Option	Default	Description
`--check`	`false`	Dry run; exit 1 if changes needed
`--diff`	`false`	Show unified diff of changes
`--delimiter-style`	`###`	Target: `###`, `---`, `===`, `***`, `~~~`
`--variable-style`	`double_brace`	Target: `double_brace`, `single_brace`, `angle_bracket`
`--max-line-length`	`120`	Max line length (0 to disable)
`--quiet`, `-q`	`false`	Suppress non-error output

`promptfmt init`

Generate a default `.promptfmt.yaml` in the current directory.

Formatting Pipeline

Rules are applied in this order:

1. **Whitespace normalization** -- LF line endings, strip trailing whitespace, collapse blank lines, ensure final newline
2. **Delimiter normalization** -- Normalize `---`, `===`, `***`, `~~~` to a single style (code blocks preserved)
3. **Variable syntax normalization** -- Convert `{{var}}`, `{var}`, `<var>` to target style (inline code preserved)
4. **Line wrapping** -- Wrap at word boundaries; skip URLs, tables, headings, code blocks
5. **Structure normalization** (YAML/JSON only) -- Sort keys, consistent indentation, re-serialize

After formatting, promptfmt re-parses the output and verifies semantic equivalence. If formatting changes the meaning of the prompt, the change is rejected with an error.

Semantic Equivalence

Two prompts are equivalent if:

- Same number of messages
- Each message has the same role
- Message content is identical after whitespace normalization
- Same variable names (syntax style may differ)
- Same metadata keys and values

Programmatic API

```
from promptfmt import format_file, format_content, FmtConfig, is_equivalent
```

```python
config = FmtConfig(
    delimiter_style="###",
    variable_style="double_brace",
    max_line_length=120,
    wrap_style="soft",
    sort_metadata_keys=True,
    indent=2,
)

# Format a file
result = format_file("prompts/greeting.yaml", config)
# result.changed, result.equivalent, result.formatted_content, result.error

# Format raw content
from promptfmt_core import PromptFormat
formatted = format_content(raw_content, PromptFormat.YAML, config)
```

promptcost Reference

CLI Commands

`promptcost estimate <path>`

Option	Default	Description
`--model`, `-m`	`claude-4-sonnet`	Model profile for estimation
`--output-tokens`	auto	Override estimated output tokens
`--project`	none	Volume projection (e.g., `1000/day`)
`--compare`	`false`	Compare across default model set
`--models`	none	Comma-separated model list for comparison
`--format`, `-f`	`text`	Output: `text` or `json`

Default comparison models: `gpt-4o`, `gpt-4o-mini`, `claude-4-sonnet`, `gemini-2.0-flash`.

`promptcost budget <path>`

Option	Required	Description
`--limit`, `-l`	yes	Max cost per invocation in USD
`--model`, `-m`	no	Model profile (default: `claude-4-sonnet`)
`--output-tokens`	no	Override output token estimate

`promptcost delta <old_path> <new_path>`

Show cost impact of a prompt change.

Option	Description
`--model`, `-m`	Model profile
`--volume`	Volume for monthly impact projection
`--output-tokens`	Override output tokens

`promptcost models`

List all available model profiles with pricing.

Output Token Estimation Heuristics

When `--output-tokens` is not specified:

Priority	Source	Method
1	`--output-tokens` CLI flag	explicit

2	`expected_output_tokens` in prompt metadata	explicit
3	Content keywords: `json`, `structured`, `schema`, `format`	heuristic (500 tokens)
4	Content keywords: `brief`, `short`, `concise`, `summary`	heuristic (300 tokens)
5	Content keywords: `detailed`, `comprehensive`, `thorough`, `essay`	heuristic (2,000 tokens)
6	Default	heuristic (1,000 tokens)

Estimates are capped at the model's `max_output_tokens`.

Volume Projection Formats

Format	Conversion
`N/hour`	N * 24 calls/day
`N/day`	N calls/day
`N/week`	N / 7 calls/day
`N/month`	N / 30 calls/day

Projections: daily, monthly (daily * 30), annual (daily * 365).

Pipeline Cost Estimation

For multi-stage pipelines, each stage's estimated output tokens are added to the next stage's input context, modeling cascading token accumulation.

```
from promptools_core import parse_pipeline
from promptcost import estimate_pipeline

pipeline = parse_pipeline("pipeline.yaml")
estimate = estimate_pipeline(pipeline, model="gpt-4o")
print(f"Total pipeline cost: ${estimate.total_cost:.4f}")
for stage in estimate.stages:
    print(f"  {stage.stage_name}: ${stage.total_cost:.4f}")
```

Programmatic API

```
from promptools_core import parse_file
from promptcost import estimate_file, compare_models, project_cost, check_budget

prompt = parse_file("prompts/greeting.yaml")

# Single estimate
est = estimate_file(prompt, model="gpt-4o", output_tokens=500)
# est.input_tokens, est.estimated_output_tokens, est.total_cost

# Model comparison
comp = compare_models(prompt, ["gpt-4o", "gpt-4o-mini", "claude-4-sonnet"])
```

```
# comp.cheapest, comp.most_expensive, comp.savings_vs_most_expensive

# Volume projection
proj = project_cost(est, "1000/day")
# proj.daily_cost, proj.monthly_cost, proj.annual_cost

# Budget check
results = check_budget([est], budget=0.05)
# results[0].over_budget, results[0].overage
```

prompttest Reference

CLI Commands

`prompttest run <path>`

Option	Default	Description
`--format`, `-f`	`text`	Output: `text`, `json`, `junit`
`--model`, `-m`	none	Override model for cost/token assertions
`--fail-fast`	`false`	Stop after first failure
`--verbose`, `-v`	`false`	Detailed output for all tests
`--pattern`, `-p`	`test_*.yaml`	Glob pattern for test file discovery

`prompttest init`

Create an example `test_example.yaml` in the current directory.

Test File Format

```yaml
suite: suite-name              # optional, defaults to filename
prompt: path/to/prompt.yaml    # required, relative to test file
model: gpt-4o                  # optional, default model for assertions

tests:
  - name: test-name
    assert: assertion_type
    # assertion-specific parameters...
    skip: false                # optional
    skip_reason: "reason"      # optional
    case_sensitive: false      # optional, for text/regex assertions
    model: gpt-4o             # optional, override suite model
```

All 15 Assertion Types

Content assertions

Type	Required Params	Description
`contains`	`text`	Content contains text
`not_contains`	`text`	Content does not contain text
`matches_regex`	`pattern`	Content matches regex
`not_matches_regex`	`pattern`	Content does not match regex

Structure assertions

Type	Required Params	Description

`has_role`	`role`	Has message with given role
`has_variables`	`variables` (list)	Uses specific template variables
`has_metadata`	`keys` (list)	Has specific metadata keys
`valid_format`	none	File parses with at least one message

Token/size assertions

Type	Required Params	Description
`max_tokens`	`max`	Total tokens under maximum
`min_tokens`	`min`	Total tokens above minimum
`max_messages`	`max`	Message count under maximum
`min_messages`	`min`	Message count above minimum
`token_ratio`	`ratio_max`	System/user token ratio within limit

Cost assertions

Type	Required Params	Description
`max_cost`	`max`, model	Estimated cost under budget ceiling

Regression assertions

Type	Required Params	Description
`content_hash`	`hash` (optional)	SHA256 hash matches expected value

Output Formats

Text: Rich-formatted terminal output with PASS/FAIL/ERR/SKIP indicators and summary.

JSON: Structured JSON with `total`, `passed`, `failed`, `errors`, `skipped`, `duration_ms`, and `suites` array containing per-test results.

JUnit XML: Standard JUnit format compatible with all major CI systems.

Programmatic API

```
from promptest import (
    load_test_suite, run_test_suite, run_test_file,
    run_test_directory, discover_test_files,
    format_text, format_json, format_junit,
    run_assertion, TestCase, AssertionType,
)

# High-level
report = run_test_file("tests/test_greeting.yaml")
```

```python
report = run_test_directory("tests/", fail_fast=True)

# Low-level
suite = load_test_suite("tests/test_greeting.yaml")
results = run_test_suite(suite)

# Test discovery
files = discover_test_files(Path("tests/"), pattern="test_*.yaml")

# Formatting
print(format_text(report))
junit_xml = format_junit(report)
```

Configuration Reference

Config File Locations

Config files are discovered by walking up the directory tree from the target file or current directory. At each level, these filenames are checked in order:

1. `.prompt{tool_name}.yaml` -- Tool-specific (e.g., `.promptfmt.yaml`, `.promptcost.yaml`)
2. `.promptlint.yaml` -- Suite-wide
3. `.promptlint.yaml` -- Backward compatibility

Merge Priority

1. CLI flags (highest)
2. Config file
3. Model profile defaults
4. Built-in defaults (lowest)

Full Config File Reference

```yaml
# .promptlint.yaml -- Suite-wide configuration

# Default model for all tools
model: gpt-4o

# File exclusion patterns (glob syntax)
exclude:
  - "vendor/**"
  - "*.generated.*"
  - "node_modules/**"

# Plugin directories
plugins:
  - "./my-plugins"

# Caching
cache:
  enabled: true
  dir: .promptlint-cache

# Tokenizer override
tokenizer:
  encoding: cl100k_base

# Tool-specific sections
fmt:
  delimiter_style: "###"
  variable_style: double_brace
  max_line_length: 120
  wrap_style: soft
  sort_metadata_keys: true
  indent: 2
```

Tool-Specific Config Files

`.promptfmt.yaml`

```yaml
delimiter_style: '###'
variable_style: double_brace
max_line_length: 120
wrap_style: soft
sort_metadata_keys: true
indent: 2
exclude:
  - 'vendor/**'
```

CI/CD Integration Guide

GitHub Actions

```yaml
name: Prompt Quality

on: [push, pull_request]

jobs:
  prompt-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install tools
        run: |
          pip install promptfmt promptcost promptttest

      - name: Check formatting
        run: promptfmt format prompts/ --check

      - name: Check budget
        run: promptcost budget prompts/ --limit 0.10 --model gpt-4o

      - name: Run tests
        run: promptttest run tests/ --format junit > prompt-test-results.xml

      - name: Upload test results
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: prompt-test-results
          path: prompt-test-results.xml
```

GitLab CI

```yaml
prompt-quality:
  image: python:3.12
  before_script:
    - pip install promptfmt promptcost promptttest
  script:
    - promptfmt format prompts/ --check
    - promptcost budget prompts/ --limit 0.10 --model gpt-4o
    - promptttest run tests/ --format junit > report.xml
  artifacts:
```

```
    reports:
      junit: report.xml
```

Pre-commit Hooks

```
#!/bin/sh
# .git/hooks/pre-commit

# Check formatting
promptfmt format prompts/ --check || {
    echo "Prompt formatting check failed. Run: promptfmt format prompts/"
    exit 1
}

# Check budget
promptcost budget prompts/ --limit 0.10 --model gpt-4o || {
    echo "Prompt cost exceeds budget."
    exit 1
}

# Run tests
promptfmt run tests/ || {
    echo "Prompt tests failed."
    exit 1
}
```

Exit Codes Summary

Tool	Code 0	Code 1	Code 2
promptfmt	All files formatted	Files need formatting (`--check`)	Errors occurred
promptcost	Within budget	Over budget	Path not found
promptfmt	All tests passed	Tests failed/errored	Path not found

Plugin Development Guide

The plugin system allows extending promptlint with custom logic. Plugins are Python files containing classes that subclass a tool-specific base class.

Creating a Plugin

1. Create a Python file in a plugin directory:

```python
# my_plugins/custom_rule.py
from promptlint.rules.base import BaseRule
from promptlint.models import LintConfig, LintViolation, PromptFile, Severity


class NoTodoComments(BaseRule):
    rule_id = "custom/no-todo"
    name = "No TODO comments in prompts"
    default_severity = Severity.WARNING
    fixable = False

    def check(self, prompt_file, config):
        violations = []
        for msg in prompt_file.messages:
            if "TODO" in msg.content:
                violations.append(LintViolation(
                    rule_id=self.rule_id,
                    message="Found TODO comment in prompt",
                    severity=self.default_severity,
                    line=msg.line_start,
                ))
        return violations
```

2. Register the plugin directory in your config:

```yaml
# .promptlint.yaml
plugins:
  - "./my_plugins"
```

Plugin Discovery Rules

- Only `.py` files are scanned (files starting with `_` are skipped)
- Classes must be concrete subclasses of the specified base class (not abstract)
- Duplicate class names across plugin files are skipped with a warning
- Plugins are loaded in sorted filename order within each directory
- Import errors are logged and the plugin is skipped (does not halt execution)

Troubleshooting

"Unsupported file extension" error

Ensure your prompt files use a supported extension: `.yaml`, `.yml`, `.json`, `.md`, or `.txt`.

"tiktoken is not installed" error

Install tiktoken: `pip install tiktoken`. This is a required dependency of promptfmt-core and should be installed automatically.

"Unknown model profile" error

The model name does not match any built-in or custom profile. Run `promptcost models` to see available profiles. You can register custom profiles programmatically with `register_profile()`.

Token counts differ from the LLM provider's count

Non-OpenAI models (Anthropic, Google) use `cl100k_base` as an approximation. Token counts may differ from the provider's actual tokenizer. The `approximate_tokenizer` flag on the profile indicates when this is the case.

promptfmt reports "Formatting altered semantic content"

This means the formatting rules changed the meaning of the prompt (e.g., altered message content or removed variables). This is a safety check. If you see this error, please report it.

Config file not being found

Config files are discovered by walking up the directory tree. Ensure the file is in the target file's directory or any parent directory. Check the filename: `.promptfmt.yaml` (with leading dot), `.promptfmt.yaml`, etc.

JUnit XML not recognized by CI

Ensure you are writing the output to a file: `promptfmt run tests/ --format junit > results.xml`. The XML is printed to stdout.

FAQ

Q: Can I use promptcost without any LLM API calls? A: Yes. All tools work entirely offline. They parse, analyze, and test prompt files without making any API calls.

Q: Which tokenizer is used for non-OpenAI models? A: Non-OpenAI models use `cl100k_base` as an approximation. This is noted in the model profile with `approximate_tokenizer=True`. Token counts may differ slightly from the provider's actual tokenizer.

Q: Can I add support for a new model? A: Yes. Use `register_profile()` to add a custom `ModelProfile` with the model's context window, pricing, and encoding.

Q: How does promptcost estimate output tokens? A: It uses keyword-based heuristics (e.g., "json" suggests ~500 tokens, "detailed" suggests ~2000 tokens). You can override this with `--output-tokens` or by setting `expected_output_tokens` in your prompt file's metadata.

Q: Can I use promptcost with pytest? A: promptcost is a standalone framework with its own YAML test files. It does not integrate directly into pytest test discovery. However, you can call promptcost's Python API from within a pytest test if desired.

Q: What happens if promptfmt changes the meaning of my prompt? A: promptfmt verifies semantic equivalence after formatting. If the formatted output is not semantically equivalent to the original, the change is rejected and an error is reported.

Q: How do I exclude files from processing? A: Add glob patterns to the `exclude` list in your config file, e.g., `exclude: ["vendor/**", "*.generated.*"]`.

Q: Can I use these tools in a monorepo with non-prompt files? A: Yes. Tools only process files with supported extensions (`.yaml`, `.yml`, `.json`, `.md`, `.txt`). Use the `exclude` config to skip directories like `node_modules` or `vendor`.

---

promptcost v1.0.0 -- Author: Scott Converse -- License: MIT